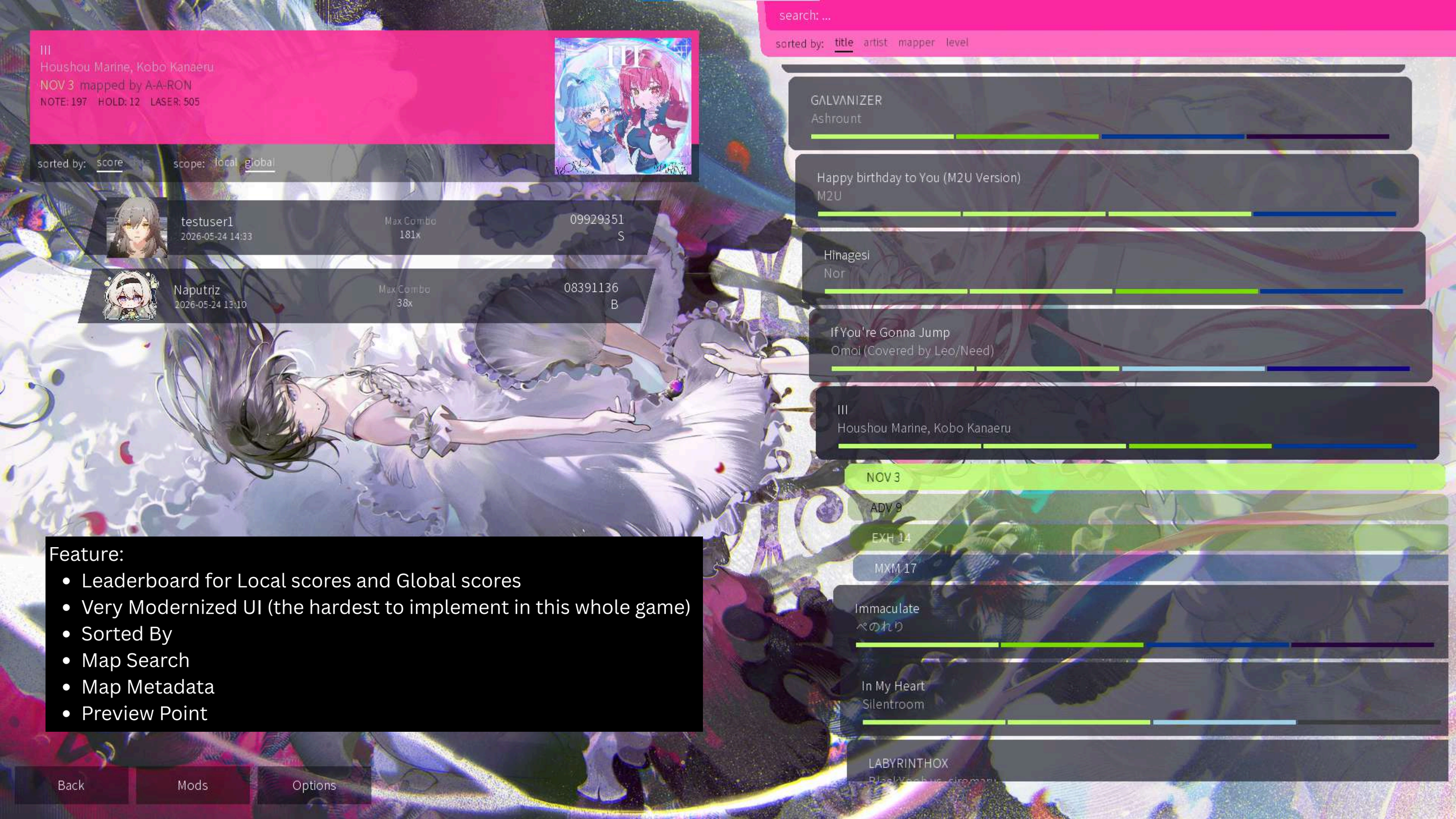
Node Vortex

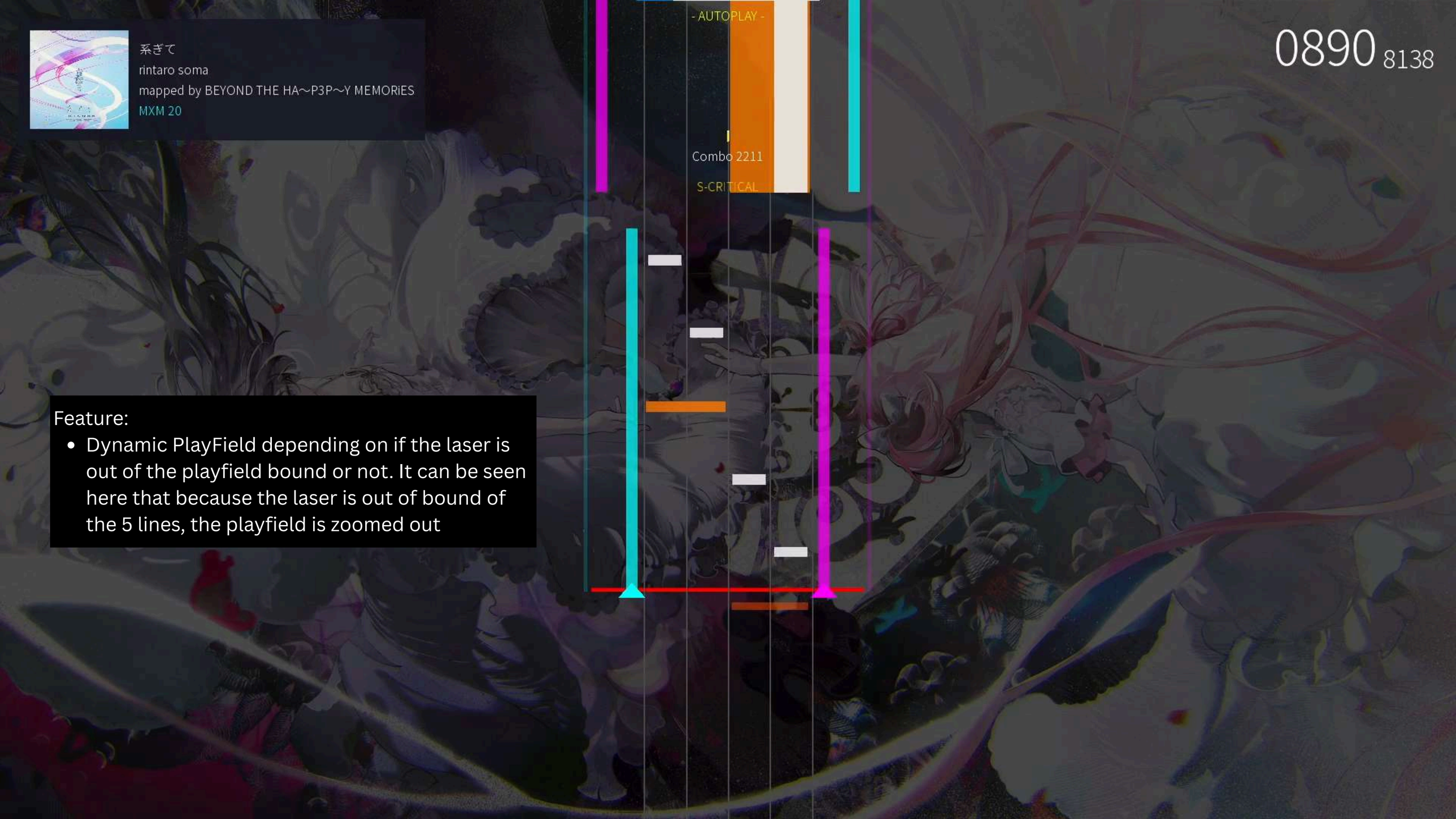
Arkaan Pasya Seplitara


Node
Vortex
START
EXIT



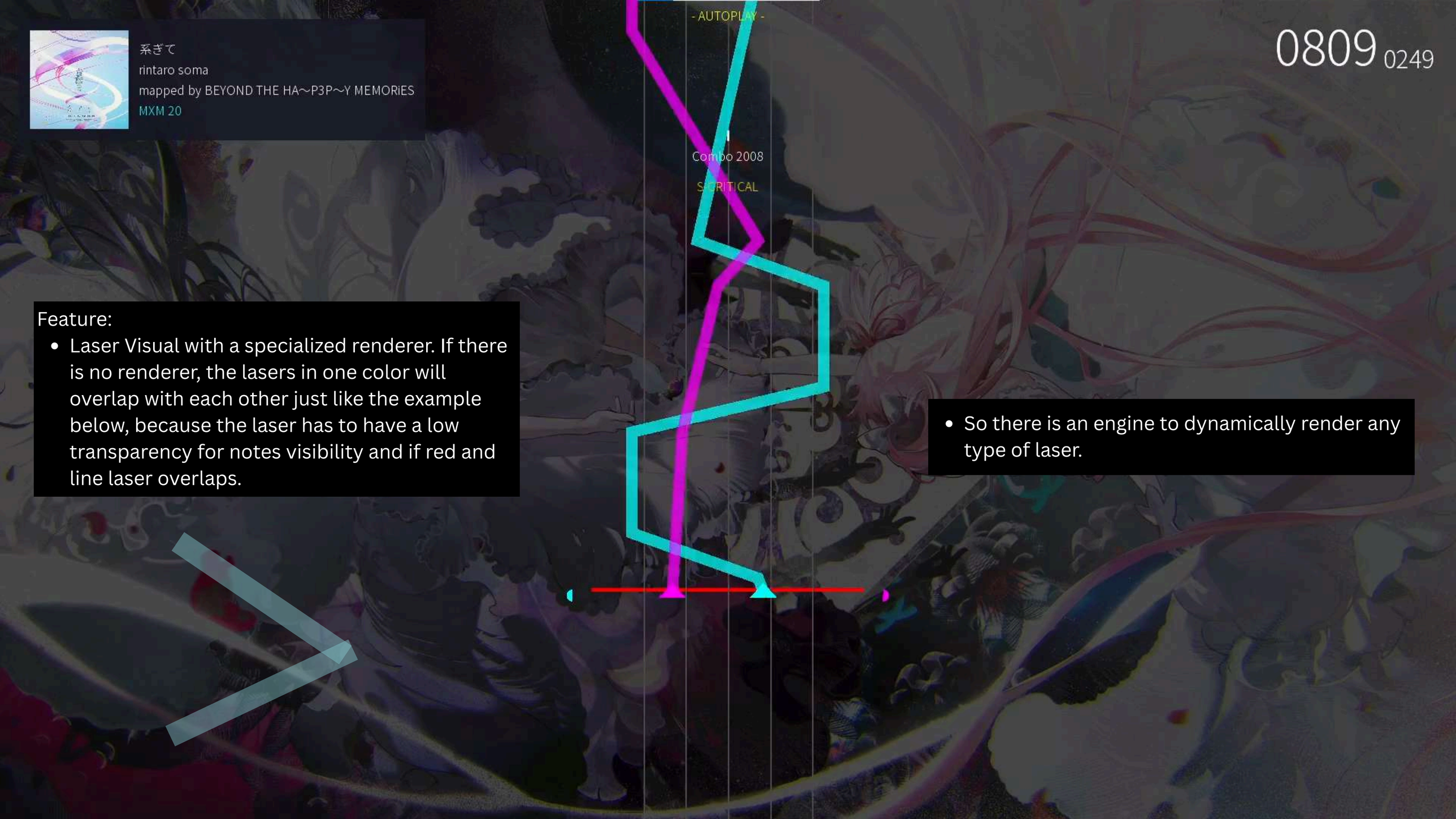
Feature:

- Leaderboard for Local scores and Global scores
- Very Modernized UI (the hardest to implement in this whole game)
- Sorted By
- Map Search
- Map Metadata
- Preview Point



Feature:

- Dynamic PlayField depending on if the laser is out of the playfield bound or not. It can be seen here that because the laser is out of bound of the 5 lines, the playfield is zoomed out



Feature:

- Laser Visual with a specialized renderer. If there is no renderer, the lasers in one color will overlap with each other just like the example below, because the laser has to have a low transparency for notes visibility and if red and line laser overlaps.

- So there is an engine to dynamically render any type of laser.

系ぎて
rintaro soma
mapped by BEYOND THE HA～P3P～Y MEMORiES

EXH 18

AA

0943 4539

S-Critical	698
Critical	362
Near	114
Mid	28
Far	5
Miss	20
Laser Tick	653
Laser Miss	27
Early	161
Late	348
Max Combo	190
Note Accuracy	93.4%
Laser Accuracy	96.0%
Volforce	16.068 (+0.000)

played on 2026-05-24 06:10

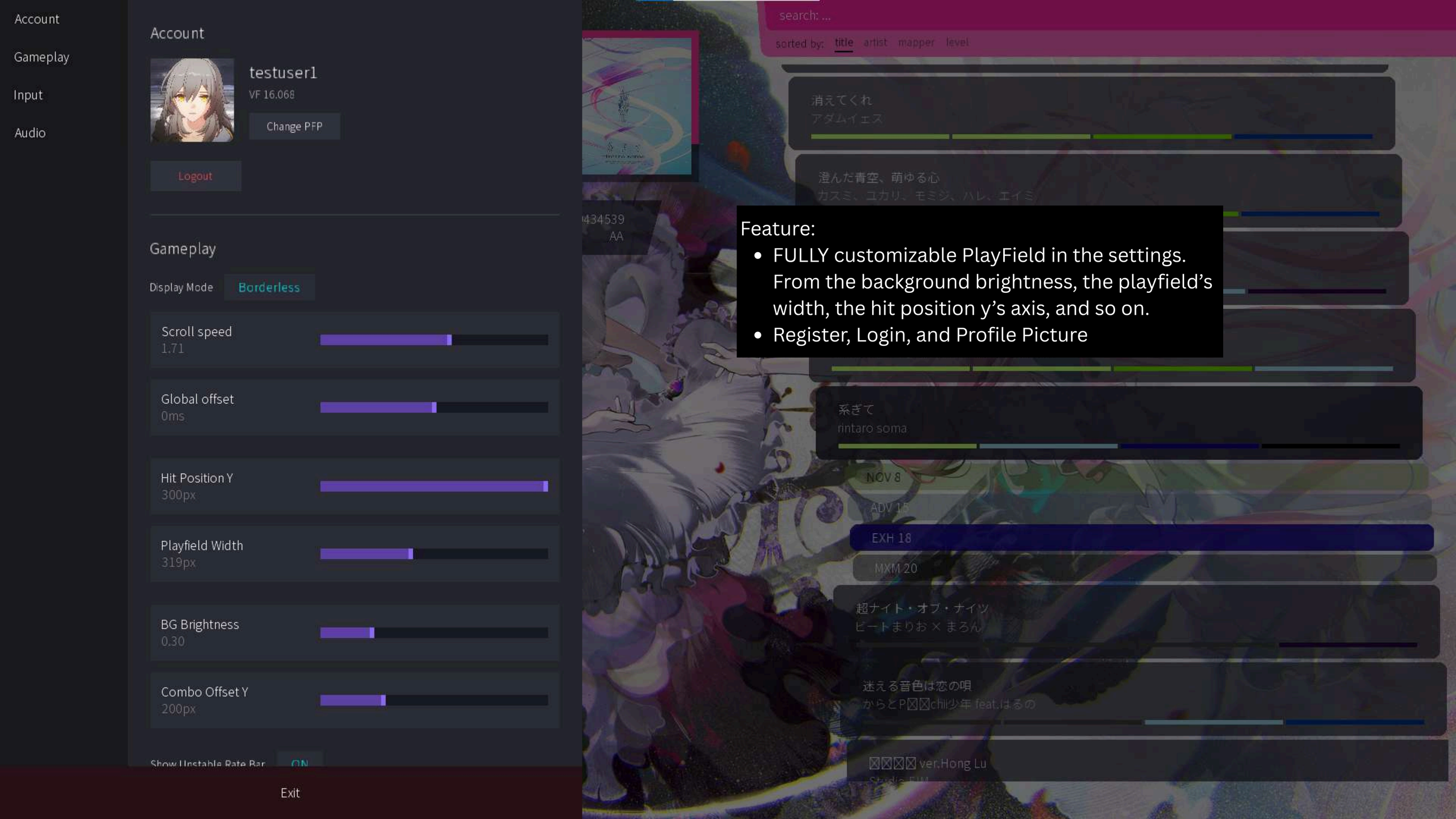
Exit

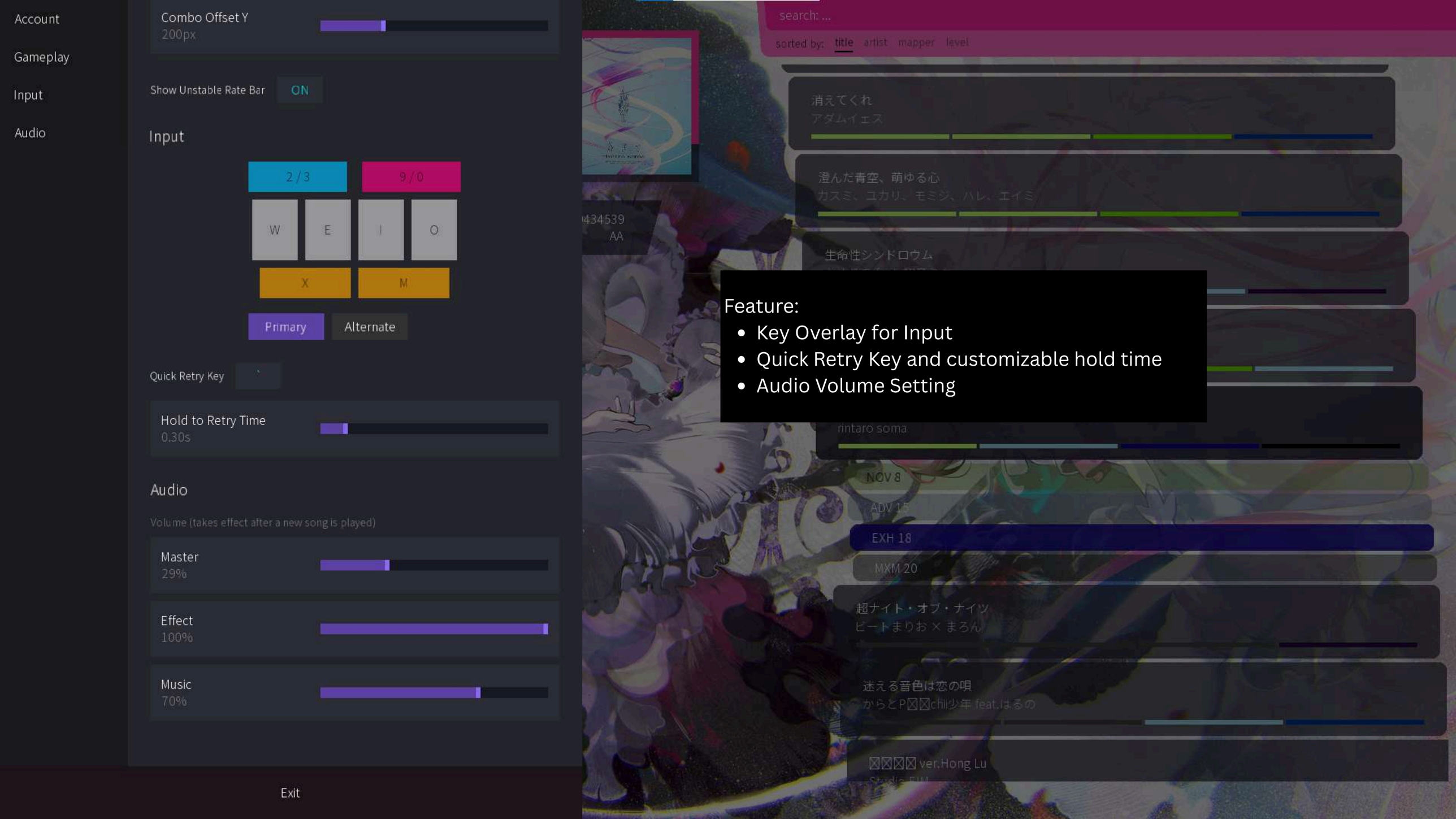
Feature:

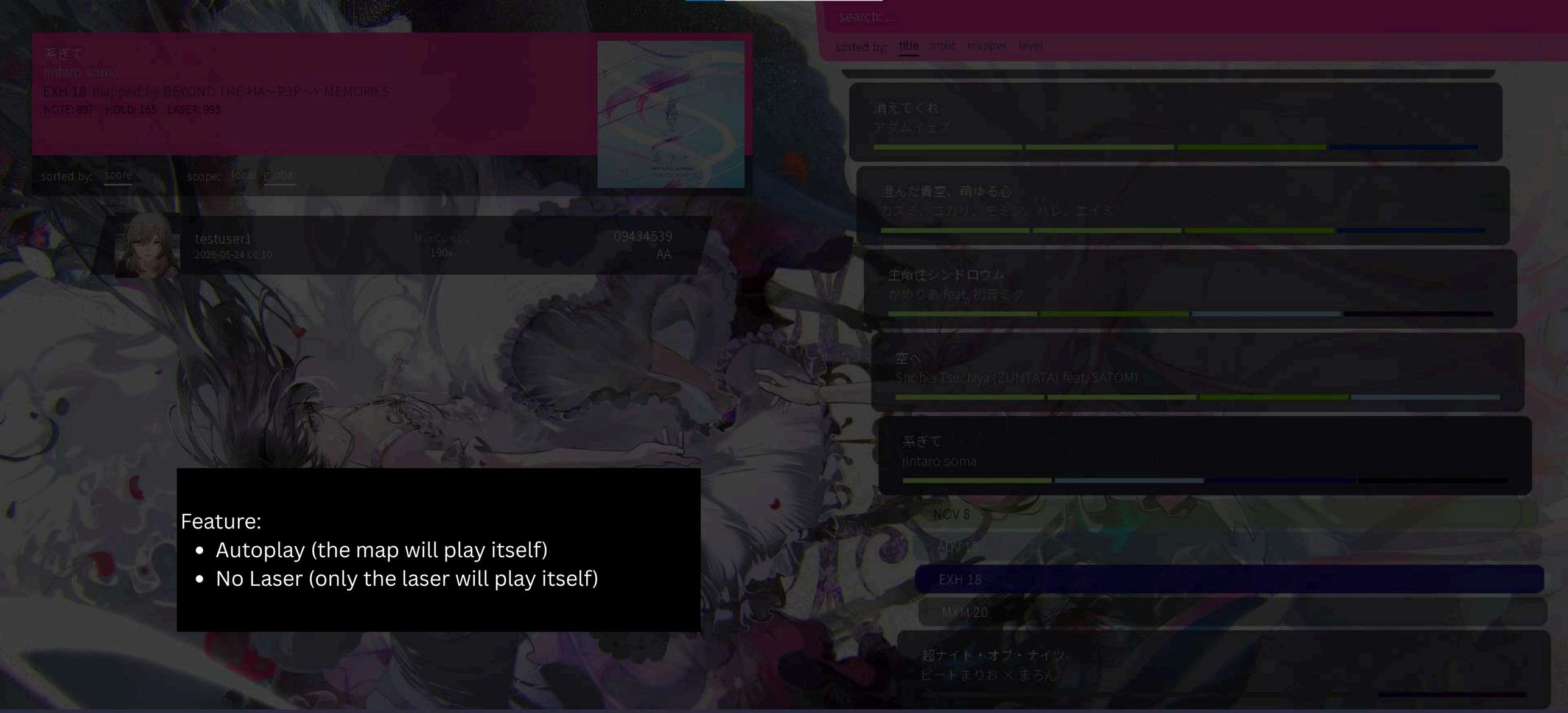
- Watch Replay Feature (can see the ‘video’ of any score in the leaderboard)
- Volforce (for player leaderboard) system

testuser1

Watch Replay


Account
Gameplay
Input
Audio
Account
testuser1
VF 16.068
Change PFP
Logout
Gameplay
Display Mode
Borderless
Scroll speed
1.71
Global offset
0ms
Hit Position Y
300px
Playfield Width
319px
BG Brightness
0.30
Combo Offset Y
200px
Show Unstable Rate Bar
ON
Exit
search: ...
sorted by: title artist mapper level
消えてくれ
アダムイエス
澄んだ青空、萌ゆる心
カスミ、ユカリ、モミジ、ハレ、エイミ
系ぎて
rintaro soma
NOV 8
ADV 15
EXH 18
MXM 20
超ナイト・オブ・ナイツ
ビートまりお × まろん
迷える音色は恋の唄
からとP□□chil少年 feat.はるの
□□□□ ver.Hong Lu
Feature:
FULLY customizable PlayField in the settings. From the background brightness, the playfield's width, the hit position y's axis, and so on.
Register, Login, and Profile Picture


Account
Gameplay
Input
Audio
Combo Offset Y
200px
Show Unstable Rate Bar
ON
Input
2 / 3
9 / 0
W
E
I
O
X
M
Primary
Alternate
Quick Retry Key
`
Hold to Retry Time
0.30s
Audio
Volume (takes effect after a new song is played)
Master
29%
Effect
100%
Music
70%
Exit
search: ...
sorted by: title artist mapper level
消えてくれ
アダムイエス
澄んだ青空、萌ゆる心
カスミ、ユカリ、モミジ、ハレ、エイミ
生命性シンドロウム
rintaro soma
NOV 8
ADV 15
EXH 18
MXM 20
超ナイト・オブ・ナイツ
ビートまりお × まろん
迷える音色は恋の唄
からとP⊠⊠chi少年 feat.はるの
⊠⊠⊠⊠ ver.Hong Lu
Feature:
Key Overlay for Input
Quick Retry Key and customizable hold time
Audio Volume Setting


search: ...
sorted by: title artist mapper level
消えてくれ
アダムイェズ
混んだ青空、萌ゆる心
カスミ、ユカリ、モミジ、ハレ、エイミ
生命性シンドロウム
かめりあ feat. 初音ミク
空へ
Shichei Tsuchiya (ZUNTATA) feat. SATOMI
系ぎて
rintaro soma
NOV 8
ADV 15
EXH 18
MXM 20
超ナイト・オブ・ナイツ
ビートまりお × まろん
系ぎて
rintaro soma
EXH 18 mapped by BEYOND THE HA~P3P~Y MEMORIES
NOTE: 897 HOLD: 165 LASER: 995
sorted by: score
scope: local global
testuser1
2026-05-24 06:10
Max Combo
190x
09434539
AA
Feature:
Autoplay (the map will play itself)
No Laser (only the laser will play itself)
Automation
Autoplay
No Laser
Exit

6 Design Pattern

- Strategy Pattern:
  - e.g. ScoreManager.java

```java
private final StrictJudgment judgmentStrategy;  2 usages

public ScoreManager(StrictJudgment strategy) { this.judgmentStrategy = strategy; }

// split Notes and Releases for the Max Score math
public void setMaxPossibleScore(int totalNotes, int totalReleases, int totalLaserTicks) {  1 usage  arkaanp
    this.maxHitScore = (totalNotes * 3.0f) + (totalReleases * 3.0f) + (totalLaserTicks * 3.0f);
    this.maxLaserTicks = totalLaserTicks;
}

public void onHit(float diffMs, String type) {  5 usages  arkaanp
    StrictJudgment.JudgmentResult result = judgmentStrategy.evaluateJudgment(diffMs, type);
    StatCategory stats = type.equals("RELEASE") ? releaseStats : noteStats;

    if (!result.tier.equals("MISS")) {
        combo++;
        if (combo > maxCombo) maxCombo = combo;

        // --- Record the hit for the UR Bar (Ignore lasers) ---
        if (!type.equals("LASER")) {
            recentHits.add(new HitMarker(diffMs, result.tier));
        }
    } else {
        combo = 0;
    }

    // Only update UI text for Notes/Releases, and format it clearly (e.g., "NEAR EARLY")
    if (!result.tier.equals("S-CRITICAL") && !result.tier.equals("MISS")) {
```

The `JudgmentStrategy` interface allows us to swap different scoring algorithms (like `StrictJudgment`) into the `ScoreManager`. This lets us change how "strict" the timing is without rewriting the score logic.

6 Design Pattern

- Command Pattern:
  - e.g. ReplayData.java & PlayScreen.java

```java
public class ReplayData {  13 usages  arkaanp
    public String difficulty = "";  1 usage
    public int finalScore = 0;  1 usage
    public long timestamp = 0;  1 usage

    // A chronological list of every single button press/release
    public Array<InputEvent> events = new Array<>();  8 usages

    public static class InputEvent {  arkaanp
        public float audioTimeMs;  2 usages
        public String inputType; // "BT1", "BT2", "FX_L", "LASER_L", etc.  2 usages
        public boolean isPressed; // true = Pressed down, false = Released  2 usages

        // Empty constructor needed for JSON parsing
        public InputEvent() {}  arkaanp

        public InputEvent(float audioTimeMs, String inputType, boolean isPressed) {  arkaanp
            this.audioTimeMs = audioTimeMs;
            this.inputType = inputType;
            this.isPressed = isPressed;
        }
    }
}
```

The `InputEvent` class encapsulates key presses as objects. These "commands" are stored in a list and then "executed" during replays to mimic the player's exact movements.

```java
    continueBtn.addListener(new ClickListener() { @Override public void clicked(InputEvent event, float x, float y) { resumeGame(); } });
    retryBtn.addListener((ClickListener) clicked(event, x, y) -> {  arkaanp
            if (music != null) music.stop();
            game.setScreen(new PlayScreen(game, mapFilePath,  replayTimestampToLoad: 0L,  isRetry: true));
    });
    exitBtn.addListener((ClickListener) clicked(event, x, y) -> {
            if (music != null) music.stop();
            if (game.songSelectScreen != null) {
                game.songSelectScreen.dispose();
            }
            game.songSelectScreen = new SongSelectScreen(game,  mainMenuMusic: null, mapFilePath,  slideInFromRight: true);
            game.setScreen(game.songSelectScreen);
    });
}
```

6 Design Pattern

- Object Pool Pattern:
  - e.g. Note.java & PlayScreen.java

```java
import com.badlogic.gdx.graphics.glutils.ShapeRenderer;
import com.badlogic.gdx.utils.Pool;

public class Note implements Pool.Poolable {  19 usages  new *
    private final float NOTE_THICKNESS = 20f;  2 usages
    public int lane;  11 usages
    public float startTime;  8 usages
    public float endTime;  5 usages
    public boolean isHold;  14 usages
    public boolean isActive;  3 usages

    public boolean wasHeadHit;  11 usages
    public boolean isCompleted;  11 usages
    public boolean isMissed;  9 usages

    public float y;

    public Note() {}  1 usage  new *

    public void init(int lane, float startTime, float endTime, boolean isHold) {  1 usage  new *
        this.lane = lane;
        this.startTime = startTime;
        this.endTime = endTime;
        this.isHold = isHold;

        this.isActive = true;
        this.wasHeadHit = false;
        this.isCompleted = false;
        this.isMissed = false;
    }
```

```java
// Object Pool for Notes
private Array<Note> activeNotes = new Array<>();  10 usages
private final Pool<Note> notePool = new Pool<Note>() {  2 usages  arkaanp
    @Override protected Note newObject() { return new Note(); }  arkaanp
};

private Note createNote(int lane, float start, float end, String type) {  1 usage  arkaanp
    Note note = notePool.obtain();
    note.init(lane, start, end, "HOLD".equals(type));
    return note;
}
```

To prevent lag (Garbage Collection), the game doesn't delete `Note` objects. Instead, it uses a `Pool<Note>` to recycle them. When a note goes off-screen, it is "reset" and put back in the pool for the next song.

6 Design Pattern

- Observer Pattern:
  - e.g. NetworkManager.java & TextureLoader.java

```
public interface NetworkCallback<T> {  17 usages  11 implementations  arkaanp
    void onSuccess(T result);  8 usages  11 implementations  arkaanp
    void onError(String message);  23 usages  11 implementations  arkaanp
}
```

```
public static void register(String username, String password, final NetworkCallback<String> callback) {  1 usage
    Net.HttpRequest request = new Net.HttpRequest(Net.HttpMethods.POST);
    request.setUrl(BASE_URL + "/auth/register");
    request.setHeader( name: "Content-Type",  value: "application/json");
    request.setTimeOut(20000);
```

```
public interface TextureCallback {  2 usages  1 implementation  arkaanp
    void onLoaded(Texture texture);  2 usages  1 implementation  arkaanp
    void onError(Throwable t);  4 usages  1 implementation  arkaanp
}

public static void loadTexture(String url, final TextureCallback callback) {
    url = transformCloudinaryUrl(url);
    if (url == null || url.isEmpty()) {
        callback.onError(new Exception("Empty URL"));
        return;
    }
```

The `NetworkCallback<T>` and `TextureCallback` interfaces act as observers. The UI "subscribes" to the results, and the managers "notify" the UI once the profile picture or score data has finished loading.

6 Design Pattern

- State Pattern:
  - e.g. LaserCursor.java

```
public class LaserManager {  3 usages   arkaanp
    public void updateCursor(LaserCursor cursor, Array<Beatmap.LaserSequence> laserData, float currentTime, float delta,
                }

                // --- APPLY FINAL STATE ---
                if (wrongInput) {
                    cursor.isMissed = true;
                    cursor.missedTimer += delta * 1000f;
                    cursor.setState(new com.nodevoltex.game.patterns.FreeState());

                    // They stay physically stranded wherever they fell off so they can see their mistake

                    if (cursor.missedTimer >= graceWindowMs && !cursor.hasComboBroken) {
                        scoreManager.onMiss( type: "LASER");
                        cursor.hasComboBroken = true;
                    }
                } else {
                    cursor.isMissed = false;
                    cursor.missedTimer = 0f;
                    cursor.hasComboBroken = false;
                    cursor.setState(new LockedState());
```

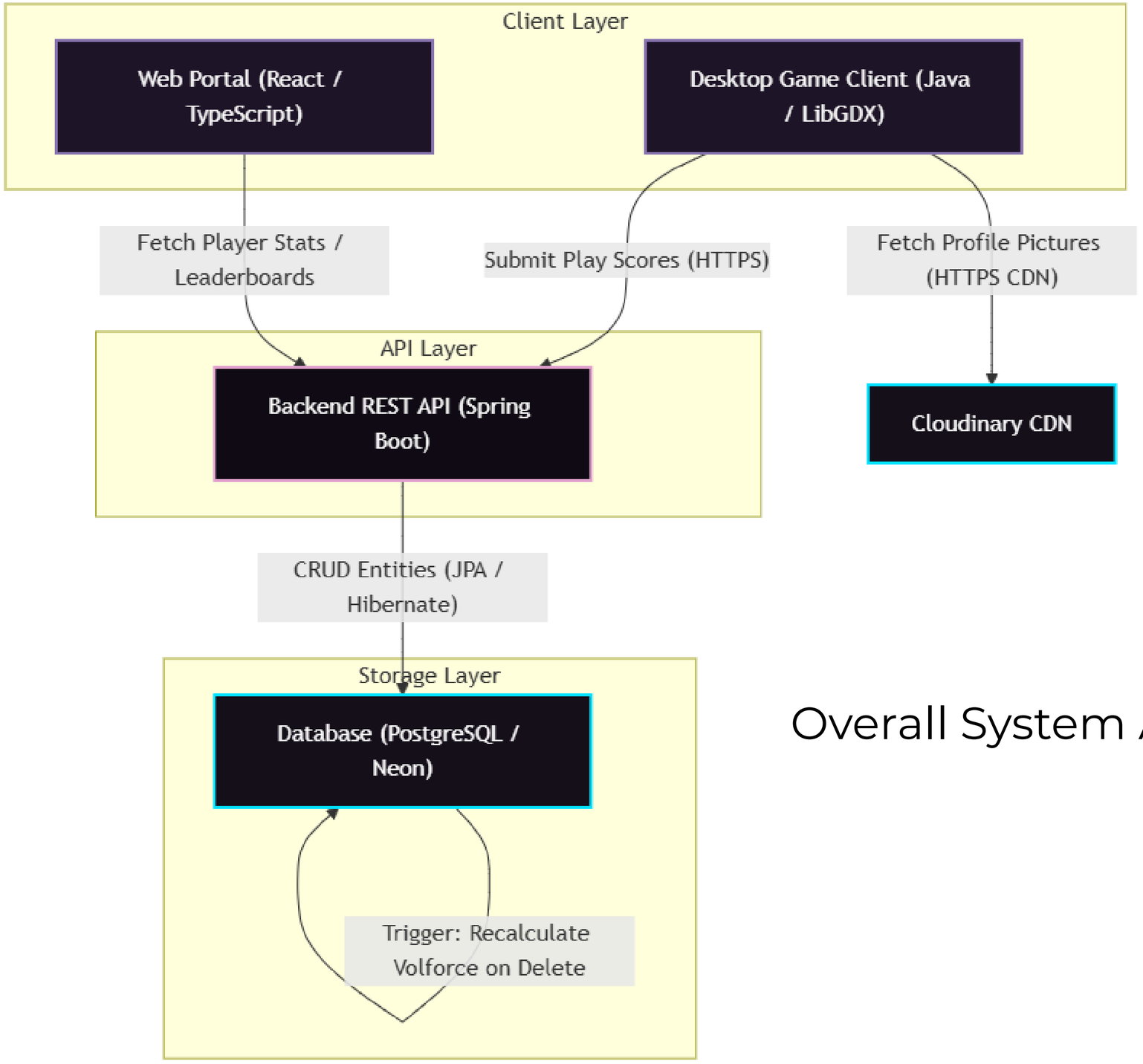
The laser cursor uses the State pattern to switch between `FreeState` (moving freely) and `LockedState` (attached to a slider). The behavior of the cursor changes completely depending on which state object is active.

6 Design Pattern

- Singleton Pattern:
  - e.g. SettingsManager.java (and Backend Services)

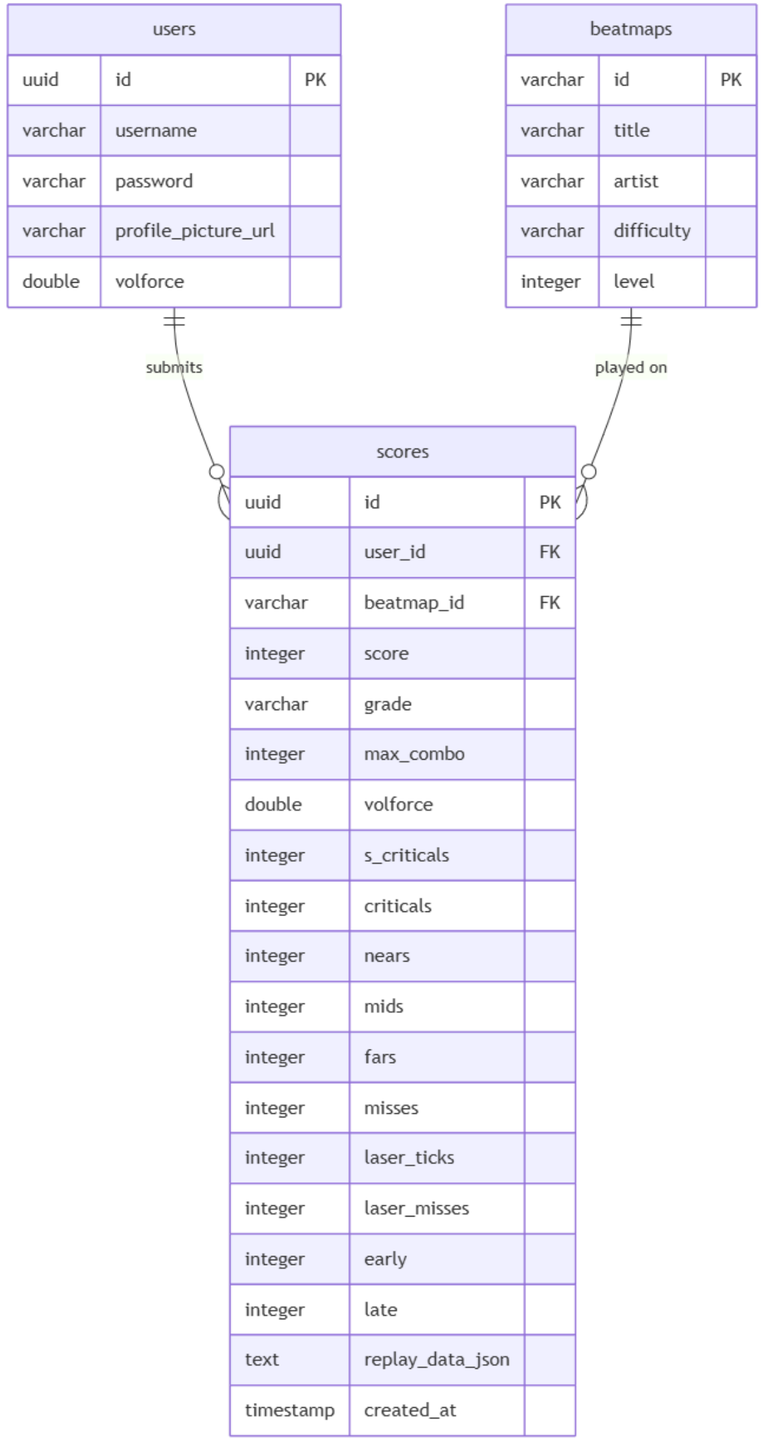
```java
package com.nodevoltex.game.managers;

import ...

public class SettingsManager {  arkaanp
    private static Preferences prefs;  3 usages

    private static Preferences getPrefs() {  60 usages  arkaanp
        if (prefs == null) prefs = Gdx.app.getPreferences( s: "NodeVoltexSettings");
        return prefs;
    }

    // --- DEFAULTS APPLIED HERE ---
    public static float getScrollSpeed() { return getPrefs().getFloat( s: "scrollSpeed",  v: 1.2f); }  3 usages
    public static float getGlobalOffset() { return getPrefs().getFloat( s: "globalOffset",  v: 0f); }  4 usages

    // Master Volume defaulted to 30%
    public static float getMasterVolume() { return getPrefs().getFloat( s: "masterVolume",  v: 0.3f); }  9 usage
    public static float getMusicVolume() { return getPrefs().getFloat( s: "musicVolume",  v: 0.8f); }  8 usages
    public static float getEffectVolume() { return getPrefs().getFloat( s: "effectVolume",  v: 1.0f); }  4 usage

    //     Playfield Customization
```

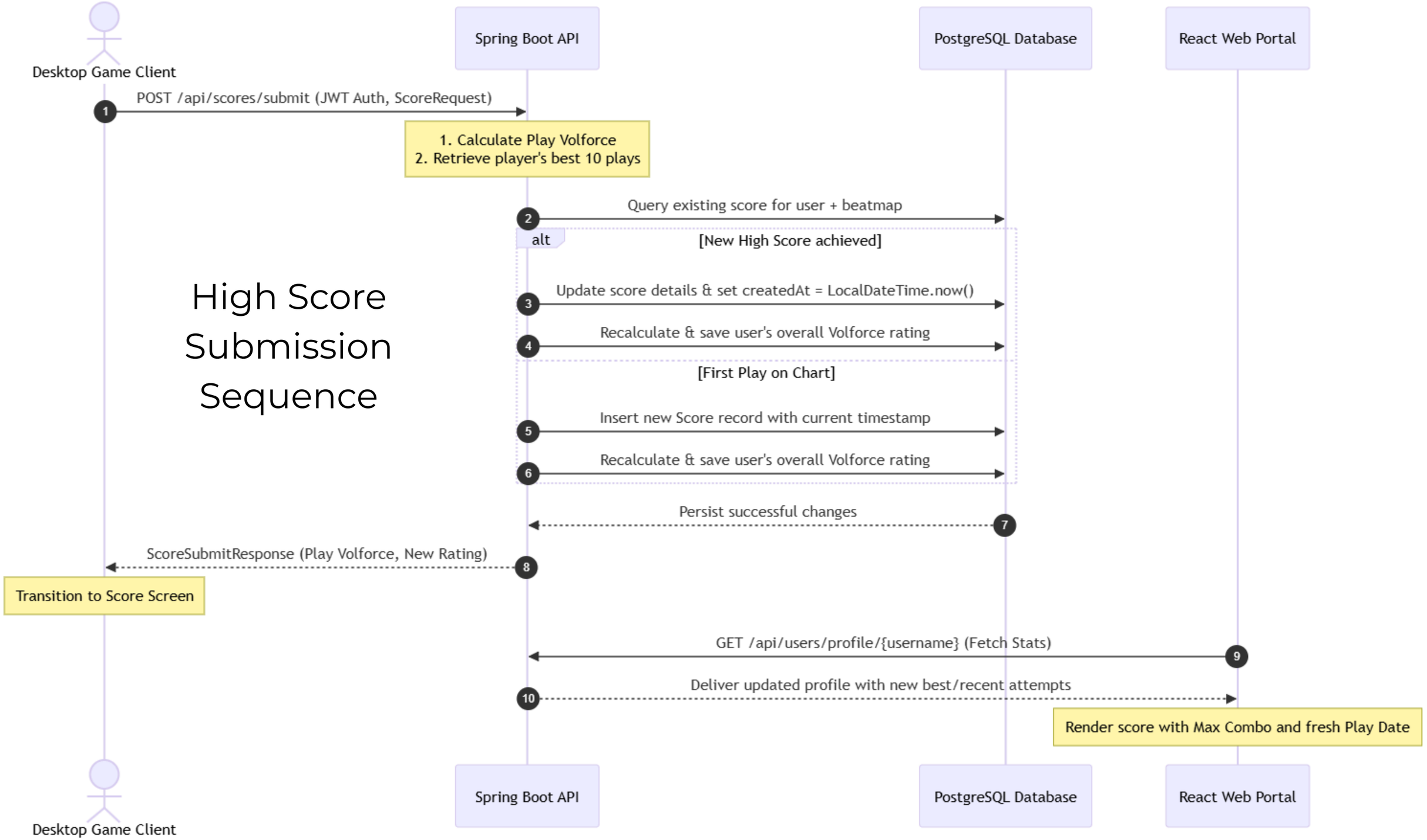
The `SettingsManager` provides a single, global point of access for all game configurations. In the Backend, Spring Boot manages your services (like `UserService`) as singletons automatically.



Overall System Architecture

Database Entity Relationship Diagram (ERD)




Desktop Game Client
Spring Boot API
PostgreSQL Database
React Web Portal
POST /api/scores/submit (JWT Auth, ScoreRequest)
1
1. Calculate Play Volforce
2. Retrieve player's best 10 plays
Query existing score for user + beatmap
2
alt
[New High Score achieved]
High Score Submission Sequence
Update score details & set createdAt = LocalDateTime.now()
3
Recalculate & save user's overall Volforce rating
4
[First Play on Chart]
Insert new Score record with current timestamp
5
Recalculate & save user's overall Volforce rating
6
Persist successful changes
7
ScoreSubmitResponse (Play Volforce, New Rating)
8
Transition to Score Screen
GET /api/users/profile/{username} (Fetch Stats)
9
Deliver updated profile with new best/recent attempts
10
Render score with Max Combo and fresh Play Date
Desktop Game Client
Spring Boot API
PostgreSQL Database
React Web Portal

Interactive Gameplay Loop & Telemetry Engine